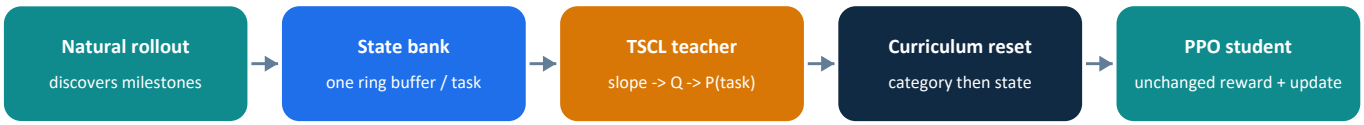
ACES/ERC -> TSCL

A direct-environment implementation guide

This report explains how the existing empirical reset curriculum was converted into a TSCL-style Window curriculum while preserving the student, environment, reward, reset-state banks, restoration logic, and training budgets. It is written to make the direct Cabinet and Factory NutThread integrations understandable from first principles.

Document status	Scope
Code state	Current workspace on 5 Oct 2026, including the corrected Factory ring-buffer counter.
Primary scope	FrankaCabinet Direct and FrankaFactory NutThread Direct.
Secondary scope	FrankaLift manager-based parity and benchmark-wide controls.
Algorithmic source	Matiisen et al. (2017), Window teacher (Algorithm 3).
Research claim	A documented TSCL-style adaptation, not an unmodified reproduction of the paper.



Two deliberately separate evidence streams

Natural episodes write empirical states. Curriculum episodes supply category-labelled TSCL outcomes.

Both streams train PPO; neither teacher statistics nor teacher reward are inserted into PPO.

Prepared as a research implementation and audit guide. Repository-relative paths and line anchors refer to the current workspace.

Contents

1. Executive summary 4

2. Original TSCL intention and what is being adapted 4

3. Exact deviations from the paper 5

4. What remains unchanged 5

5. The direct-environment lifecycle 6

6. Configuration and mode routing 6

7. Empirical state discovery and passive collection 6

8. Ring-buffer mechanics 7

9. What is stored in each direct environment 7

10. Episode labels and the two success streams 8

11. When outcomes are recorded 8

12. Teacher state and data structures 9

13. Fixed update cadence 9

14. Sparse measurement handling 9

15. Learning progress: OLS slope over real measurement times 10

16. EMA teacher value and forgetting 10

17. Bootstrap, availability, Boltzmann policy, and exploration 10

18. ACES/ERC scheduler versus TSCL scheduler 11

19. Reset allocation: what the teacher does and does not control 11

20. State selection and restoration are still ACES/ERC 12

21. Cabinet Direct: complete walkthrough 12

22. Factory NutThread Direct: complete walkthrough 13

23. Shared hyperparameters and deliberate environment-specific values 14

24. Logging and how to read TensorBoard 14

25. Final evaluation protocol 15

26. Consistency across all three tasks 15

27. Scientific caveats and interpretation limits 16

28. Implementation pseudocode 16

29. Code map 17

30. Audit checklist for future modifications 17

31. References and provenance 17

1. Executive summary

The scientific intervention is deliberately narrow: the existing ACES/ERC system already knows how to detect milestones, passively collect empirical states, store them by category, select a curriculum fraction of reset environments, restore a sampled state, and continue PPO. TSCL replaces only the rule that assigns probability mass across populated reset categories.

Component	ACES/ERC	TSCL-style implementation
State source	Natural rollouts reach milestones and write snapshots.	Unchanged.
Curriculum share	30% of resets; 70% remain natural.	Unchanged and outside the teacher.
Scheduling signal	Natural-episode difficulty and sequential bottleneck gaps.	Temporal learning-progress slope from category-labelled curriculum outcomes.
Teacher value	Difficulty/gap confidence plus soft/greedy blend.	EMA-smoothed OLS slope Q; priority is abs(Q).
Category policy	ACES distribution.	Boltzmann(abs(Q)/tau), mixed with 20% uniform exploration.
State replay	Sample one state from selected category and restore it.	Unchanged.
Student	Existing PPO, reward, observations, actions, dynamics.	Unchanged.

The central invariant
Same learner + same task + same empirical reset information + same reset share + same budget; only the category scheduler changes.

A second invariant prevents data leakage: curriculum episodes never write replayed states back into the empirical banks. Natural episodes alone populate the banks. Conversely, TSCL learning-progress measurements come only from curriculum episodes because only they carry a teacher-selected category label. Both episode types still contribute normal trajectories to PPO.

2. Original TSCL intention and what is being adapted

Matiisen et al. formulate a Teacher that chooses from a discrete set of tasks for a Student. The Teacher observes task scores and emphasizes tasks where competence changes fastest. Positive slope means learning; negative slope means forgetting; the absolute magnitude gives both cases attention. The Window algorithm stores recent scores and their observation times, fits a linear trend, and smooths that trend into a task value Q.

The paper does not define an empirical reset-state curriculum. The explicit research adaptation is therefore: one populated empirical reset category is treated as one discrete TSCL task. The Teacher chooses a category, while the pre-existing reset system chooses the physical state inside it.

```
# Paper-level Window teacher
choose task a_t from abs(Q)
train Student on a_t and observe score o_t
append (time, score) to FIFO history for a_t
r_t = slope(history[a_t])
Q[a_t] = alpha * r_t + (1 - alpha) * Q[a_t]
```

Conceptual reconstruction of Matiisen et al., Algorithm 3. Source: arXiv:1707.00183v2, Appendix A.

The paper explicitly permits choosing tasks from abs(Q) with either epsilon-greedy or a Boltzmann policy. Its experiments used alpha=0.1, temperature=0.0004, and window length K=10. Those three values are retained here. The 20% uniform mixture, fixed measurement cadence, success-rate score, minimum sample threshold, and availability mask are documented adaptations for vectorized Isaac Lab.

3. Exact deviations from the paper

Paper construct	Implementation choice	Reason and consequence
Discrete task	One ACES empirical reset category.	Preserves the existing state curriculum instead of inventing new tasks or states.
Teacher chooses one task	A distribution controls only the 30% curriculum-reset subset.	Matches massive parallelism and preserves the fixed 70% natural exposure.
Score may be episode reward	Existing binary final-task success, aggregated by category.	Comparable across environments and separate from reward scale.
One score after training a task	One success-rate point after enough completed category episodes.	Reduces noise and avoids thousands of near-duplicate measurements.
Window updated at teacher steps	Fixed PPO-iteration boundaries; sparse samples carry forward.	Keeps cadence comparable across environment counts and does not fabricate missing failure.
All tasks selectable	Only categories with at least one valid empirical state are selectable.	Prevents reading zero-initialized or invalid physical states.
Exploration via epsilon-greedy or Boltzmann	Boltzmann plus a 20% uniform mixture.	Makes starvation prevention explicit under a very low temperature.
Teacher observations	On-policy training episodes only; no separate probes.	No extra interactions or hidden compute budget.

Naming discipline
The correct description is 'TSCL-style Window curriculum.' It should not be presented as a bit-for-bit reproduction of the original experimental system.

4. What remains unchanged

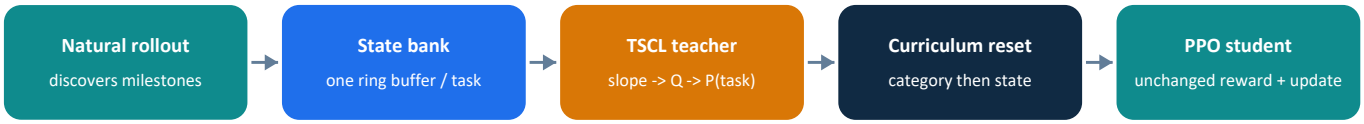
A curriculum benchmark is interpretable only if the curriculum is the independent variable. The code audit of the TSCL transition found no changes to PPO agent configuration files and no changes to reward equations, observation or action definitions, termination thresholds, physics, dynamics, episode length, state restoration, reset perturbation, or the 30% replay ratio.

Frozen layer	Evidence in the implementation
PPO / optimizer / network	No agent configuration files changed. RSL-RL remains used for Lift and Cabinet; RL-Games remains used for Factory.
Reward	Teacher values and slopes are recorded in the teacher object and logs only; no teacher quantity is added to rew_buf or returned reward.
Observations and actions	Existing observation construction and action application remain intact.
Task success	Cabinet retains drawer position > 0.39; Factory retains its existing NutThread success predicate.
Physics and dynamics	No material, controller, timestep, decimation, actuator, mass, or friction parameter is changed by TSCL.
Empirical states	Same per-category state banks, buffer capacity, snapshot format, and intra-bank uniform sampling.
Restoration and perturbation	Same joint/root-state writes and existing curriculum_dr values.
Training budget	Lift 3000 iterations; Cabinet 2500 iterations; Factory 200 RL-Games epochs.

The direct environment files necessarily receive integration hooks. 'Environment unchanged' therefore means the environment transition, reward, observation, termination, and physical reset mechanics are unchanged; it does not mean the source files are byte-identical.

5. The direct-environment lifecycle

A DirectRLEnv owns the full step and reset lifecycle. The TSCL integration uses two existing junctions: the post-physics reward path, where progress is observable every step, and _reset_idx, where the just-finished episode can be attributed before its label is overwritten.



Two deliberately separate evidence streams

Natural episodes write empirical states. Curriculum episodes supply category-labelled TSCL outcomes.

Both streams train PPO; neither teacher statistics nor teacher reward are inserted into PPO.

When	Direct-environment responsibility	TSCL responsibility
Every simulation step	Compute milestones and update per-episode progression.	If a PPO-iteration boundary is due, process accumulated outcomes and refresh probabilities.
New milestone	Capture the full replay state.	None; state discovery remains passive and natural-only.
Episode reset begins	The previous state and category label still exist.	Record success/failure for category-labelled curriculum episodes.
Reset assignment	Choose the fixed 30% curriculum subset.	Choose a category only for that subset.
State restoration	Sample state index, perturb, and write simulator state.	None; existing reset implementation remains authoritative.
PPO update	Consume normal trajectories and environment rewards.	No teacher reward or statistics enter PPO.

6. Configuration and mode routing

The mode field preserves the old scheduler as the default and makes TSCL opt-in. A teacher object is created only when both the master reset-curriculum switch and tscl_window mode are active in the direct environments. Invalid modes fail early instead of silently producing a hybrid experiment.

```
reset_state_curriculum_enabled = False  # Cabinet default
reset_curriculum_mode = "aces"         # unchanged default path

if (
    cfg.reset_state_curriculum_enabled
    and cfg.reset_curriculum_mode == "tscl_window"
):
    self.tscl_teacher = TSCLWindowTeacher(...)
```

Cabinet: direct/franka_cabinet/franka_cabinet_env.py:35-39, 347-358. Factory uses the same pattern at factory_env.py:70-91.

In ACES mode, the old _update_distribution and optional controller continue to run. In TSCL mode, the ACES scheduler and controller are bypassed, but their state-bank and reset machinery remain in use. This routing is the actual experimental switch.

7. Empirical state discovery and passive collection

The empirical banks are learned online from ordinary interaction. Each environment maintains a per-episode progression tensor. A milestone is written only on the first step where it changes from false to true. That first-crossing rule stores a physically realized state near the boundary rather than repeatedly storing every later state.

```
completions = self._get_subtasks()
world = self._get_world()
completed_before = self.progression[:, :, 0].bool()
new_completion = completions & (~completed_before)
self.progression[:, :, 0] = torch.maximum(
    self.progression[:, :, 0], completions.float()
)
```

Shared direct-environment pattern in Cabinet and Factory _update_progression().

The critical anti-contamination gate is the natural-episode mask. A replayed state often already satisfies its milestone on the first step after reset. Without this mask, curriculum states would reinsert themselves, recursively dominate the bank, and cease to represent states achieved by the policy.

```
buffer_completion = (
    new_completion
    & (~self.is_curriculum_episode).unsqueeze(1)
)
completed_envs, completed_tasks = torch.where(buffer_completion)
```

Cabinet: franka_cabinet_env.py:477-478. Factory: factory_env.py:541-542.

Why this is passive

No probe policy, scripted trajectory, Brownian expansion, goal generator, or teacher-generated state is used. The policy reaches a milestone during an ordinary rollout; the environment snapshots that exact state.

8. Ring-buffer mechanics

Each category has a fixed-capacity circular buffer of 64 states. pose_buffer_idx is the next write position. pose_buffer_count is the number of valid entries, capped at 64. The count is essential: the tensor is zero-initialized, so sampling beyond the written prefix can produce invalid robot or quaternion states.

```
for task in range(num_tasks):
    task_mask = completed_tasks == task
    if task_mask.any():
        worlds = completed_worlds[task_mask]
        count = worlds.shape[0]
        indices = (arange(count) + pose_buffer_idx[task]) % 64
        success_buffer[task, indices] = worlds
        pose_buffer_idx[task] = (pose_buffer_idx[task] + count) % 64
        pose_buffer_count[task] = min(count_so_far + count, 64)
```

Simplified equivalent of the corrected Cabinet and Factory ring-buffer write.

A Factory integration audit found that pose_buffer_count had been accidentally de-indented outside the task-match guard. That could mark unwritten entries valid or reuse an undefined count. The current workspace repairs the indentation so the count changes only when that category receives states.

9. What is stored in each direct environment

Environment	TSCL categories	Snapshot vector	Existing final success
Cabinet Direct	4: within 20 cm; within 10 cm; drawer > 0.20 m; drawer > 0.38 m.	13 values: 9 Franka joint positions + 4 cabinet joint positions.	Drawer joint > 0.39 m; success terminates the episode.
Factory NutThread Direct	3: aligned; engaged; threaded success.	23 values: 9 Franka joints + 7 held-asset root pose + 7 fixed-asset root pose.	Existing NutThread geometric/rotation success; episode still ends on synchronized timeout.

The category count and state vector differ because the physical tasks differ. This is not a teacher algorithm change: both environments present a finite category set, an availability mask, category labels, and binary outcomes to the same TSCLWindowTeacher class.

10. Episode labels and the two success streams



Natural: original initialization; eligible to add newly reached milestone states to buffers.

Curriculum: TSCL selects a populated category; existing replay code samples/restores a state.

The split is outside the teacher. TSCL redistributes only the orange portion.

curriculum_subtask is initialized to -1. During a curriculum reset it receives the sampled category. The label survives for the whole episode and is consumed at the next reset before labels and progression are cleared. The teacher treats -1 as ordinary/default and ignores it.

```
valid = (task_ids >= 0) & (task_ids < self.num_tasks)
if not valid.any():
    return
self._completed += bincount(task_ids[valid])
self._successful += bincount(task_ids[valid][successes[valid]])
```

utils/tscl_window.py:104-126. Natural episodes carry task_id=-1 and do not update TSCL histories.

Signal	Natural episodes	Curriculum episodes	Purpose
Empirical state-bank writes	Yes	No	Keep banks grounded in newly achieved policy states.
Legacy ACES difficulty / margin	Yes	No	Avoid inflated competence from partial-state starts.
TSCL category success	No	Yes	Measure performance conditional on a teacher-selected task.
PPO trajectories	Yes	Yes	Maintain the unchanged training interaction budget.
General diagnostic logs	Sometimes separated; some raw metrics include both	Sometimes separated	Debugging only; not final evaluation.

A subtle but important point

Natural-only state collection and curriculum-only TSCL scoring are not contradictory. They answer different questions: 'what states did the policy reach naturally?' and 'how capable is the student when trained from the category the teacher selected?'

11. When outcomes are recorded

The direct environments record the just-finished episode at the very beginning of _reset_idx. This ordering is mandatory. Recording after reset would read cleared labels or newly initialized simulator state and attribute the wrong outcome.

```
def _reset_idx(self, env_ids):
    if self.tscl_teacher is not None:
        successes = existing_success_test(env_ids)
        self.tscl_teacher.record_episode_outcomes(
            self.curriculum_subtask[env_ids], successes
        )
    super()._reset_idx(env_ids)
    # construct and apply the next episode's reset
```

Cabinet uses drawer joint > 0.39. Factory uses overall_success populated by the existing NutThread criterion.

There is no TSCL-specific early termination. Cabinet already terminates on its native success condition. Factory keeps its synchronized timeout behavior, so success is remembered as an episode outcome without changing when the episode ends.

12. Teacher state and data structures

TSCLWindowTeacher is environment-agnostic. It stores only category-level statistical state on the configured device. It never holds simulator objects, observations, rewards, actions, or policy parameters.

Field	Meaning
performance_history[i]	FIFO deque of valid success-rate measurements for category i; maximum length 10.
time_history[i]	PPO iteration corresponding to each stored success rate.
_completed / _successful	Outcomes accumulated since the most recent teacher boundary.
_pending_completed / _pending_successful	Sparse outcomes carried across boundaries until at least 8 episodes exist.
raw_slopes[i]	Most recently fitted OLS learning-progress slope.
q_values[i]	EMA-smoothed learning-progress value.
distribution[i]	Current probability among available empirical categories.
_reset_draws[i]	Actual category draws for diagnostics; does not affect optimization.

The teacher is deliberately a plain utility object rather than another reinforcement-learning agent. There is no teacher neural network, optimizer, replay buffer, or teacher rollout. This follows the paper's heuristic non-stationary-bandit interpretation of the Window algorithm.

13. Fixed update cadence

common_step_counter counts vector-environment steps. Dividing it by the unchanged rollout horizon produces a PPO-iteration index. update_if_due is called every environment step, but it returns immediately unless enough PPO iterations have elapsed. This avoids missing a boundary when resets are sparse.

```
ppo_iteration = (
    self.common_step_counter
    // self.cfg.tscl_rollout_steps_per_iteration
)
teacher.update_if_due(
    ppo_iteration,
    self.common_step_counter * self.num_envs,
    self.pose_buffer_count > 0,
)
```

Direct integration in Cabinet and Factory _update_progression().

Task	Student runner	Budget	Rollout horizon	Teacher interval	Boundaries
Lift	RSL-RL	3000 iterations	24	30 iterations	100
Cabinet Direct	RSL-RL	2500 iterations	16	25 iterations	100
Factory NutThread	RL-Games	200 epochs	128	2 epochs	100

The numeric interval differs because the native training budgets differ; the ratio is identical. This is the chosen meaning of 'same teacher schedule' across runners: 100 scheduled update opportunities over the unchanged training budget.

14. Sparse measurement handling

At a teacher boundary, interval outcomes are added to pending totals. A category produces a new success-rate point only when pending_completed >= 8. If fewer than 8 outcomes exist, they remain pending. They are not discarded and are not treated as failures. Consequently, the time between history points may exceed one teacher interval, which is why the true PPO iteration is stored alongside each score.

```
pending_completed += interval_completed
pending_successful += interval_successful
if pending_completed[task] < min_samples:
    continue
success_rate = pending_successful[task] / pending_completed[task]
performance_history[task].append(success_rate)
time_history[task].append(ppo_iteration)
pending_completed[task] = pending_successful[task] = 0
```

Simplified from utils/tscl_window.py:155-181.

15. Learning progress: OLS slope over real measurement times

For each task with at least five history points, ordinary least squares estimates the trend of success rate y against PPO iteration x. Centering x and y gives the scalar slope below. Using the recorded times rather than evenly spaced indices matters because sparse categories can skip update boundaries.

Slope equation

slope_i = sum_j[(x_j - mean(x)) (y_j - mean(y))] / sum_j[(x_j - mean(x))^2]

```
x = tensor(times, dtype=float64)
y = tensor(scores, dtype=float64)
x_centered = x - x.mean()
denominator = x_centered.square().sum()
slope = (x_centered * (y - y.mean())).sum() / denominator
```

utils/tscl_window.py:265-273. A zero denominator returns slope 0.

Observed trend	Slope	TSCL interpretation
Success rises	Positive	The student is learning this category; allocate attention.
Success falls	Negative	The student may be forgetting; revisit the category.
Success is flat	Near zero	Mastered, stalled, or currently unlearnable; less informative progress.

16. EMA teacher value and forgetting

The raw slope is noisy, so the teacher updates an exponentially smoothed value. With alpha=0.1, the new slope contributes 10% and the previous Q contributes 90%. Selection uses abs(Q), not Q, so a strong negative trend can receive the same priority as equally strong improvement.

Teacher value

Q_i <- 0.1 * slope_i + 0.9 * Q_i and priority_i = abs(Q_i)

This is the conceptual break from ACES/ERC. ACES asks where sequential competence is weak or bottlenecked. TSCL asks where competence is changing. A hard but perfectly flat task can be attractive to ACES and unattractive to TSCL; a previously mastered task that is being forgotten becomes attractive to TSCL.

17. Bootstrap, availability, Boltzmann policy, and exploration

A task is available only if its empirical bank contains at least one written state. Before every currently available task has five measurements, sampling is uniform over available tasks. This avoids pretending Q=0 is a measured plateau. If a new category becomes available later, bootstrap becomes uniform again until that new category also has five measurements.

```
if any(history_length[i] < 5 for i in available_tasks):
    return uniform_over_available

logits = abs(Q) / 0.0004
logits[~available_tasks] = -inf
boltzmann = softmax(logits)
P = 0.80 * boltzmann + 0.20 * uniform_over_available
```

Equivalent of utils/tscl_window.py:189-212.

The uniform mixture gives every available task a lower-bound probability of 0.20/K among K available categories. For three tasks the floor is about 0.0667; for four it is 0.05. The remainder can still concentrate sharply because tau=0.0004 is low.

18. ACES/ERC scheduler versus TSCL scheduler

Both schedulers read the same progression structure and write self.distribution. Their difference is the statistical meaning of that vector.

Stage	ACES/ERC path	TSCL path
Measurement population	Natural episodes only.	Category-labelled curriculum episodes only.
Competence statistic	EMA success by milestone.	Interval success rate in a FIFO history.
Primary signal	difficulty = 1 - success; adjacent difficulty gaps.	OLS slope of success over PPO iterations.
Temporal memory	EMA of current success.	FIFO time series plus EMA of slope Q.
Forgetting	Not the defining selection signal.	Negative slope is prioritized through abs(Q).
Selection policy	Soft/greedy distribution based on gap confidence and margin.	Boltzmann over abs(Q) plus 20% uniform.
Physical reset	Existing category/state selection and restoration.	Exactly the same physical reset path.

The ACES function remains in the environment because ACES must remain runnable. In TSCL mode, mode routing prevents it from updating the category distribution. This is preferable to deleting the old code because the same executable environment can serve both curriculum conditions.

19. Reset allocation: what the teacher does and does not control



Natural: original initialization; eligible to add newly reached milestone states to buffers.

Curriculum: TSCL selects a populated category; existing replay code samples/restores a state.

The split is outside the teacher. TSCL redistributes only the orange portion.

At each reset batch, the direct environments choose exactly int(batch_size * 0.3) entries using a random permutation. If no category has a valid state, the curriculum mask is cleared and all resets remain natural. The teacher never changes the 0.3 ratio and never selects the natural/default reset as another arm.

```
num_curriculum = int(len(env_ids) * sampling_ratio)
picked = first_n(randperm(len(env_ids)), num_curriculum)
valid_subtasks = pose_buffer_count > 0
if not valid_subtasks.any():
    picked[:] = False
subtasks = multinomial(teacher_distribution, picked.sum())
```

Simplified from Cabinet _reset_idx at lines 774-823 and Factory _reset_idx at lines 911-952.

The 30% is exact within a reset batch before availability fallback, not a Bernoulli draw per environment. Across asynchronous Cabinet resets it remains approximately 30% globally; Factory resets are synchronized.

20. State selection and restoration are still ACES/ERC

After TSCL samples categories, the direct environment samples an integer state index uniformly from the valid written prefix of each selected bank. The teacher has no access to this index and therefore cannot prefer individual physical states within a category.

```
max_valid = pose_buffer_count[subtasks].clamp(min=1)
world_ids = (rand(num_picked) * max_valid.float()).long()
worlds = success_buffer[subtasks, world_ids]
# Existing environment-specific restoration follows.
```

Shared direct-environment state sampling pattern.

Cabinet overwrites Franka and cabinet joint positions, adds the existing +/-0.02 robot-joint curriculum perturbation, resets velocity to zero, and reanchors the action integrator. Factory overwrites Franka joints and held/fixed root poses using the existing environment-origin handling; its curriculum_dr is 0.00.

21. Cabinet Direct: complete walkthrough

Cabinet is the clearest direct-environment example because episodes reset independently and success also terminates the episode. The full TSCL path is:

- Configuration selects tscl_window while preserving 16-step RSL-RL rollouts, 16,384 environments, and 2,500 iterations.
- The constructor creates a four-task TSCLWindowTeacher and leaves the old ACES fields in place.
- Every reward call refreshes intermediate kinematics, then _update_progression detects first milestone crossings.
- Natural milestone crossings write 13-value snapshots into one of four 64-slot ring buffers.
- At every 25 PPO iterations, accumulated category outcomes may create history points and update Q/probabilities.
- At reset entry, drawer > 0.39 is recorded against the prior curriculum_subtask label.
- The existing reset code constructs default robot/cabinet states for every resetting environment.
- Exactly 30% of the batch is marked curriculum when any bank is available.
- TSCL supplies the category distribution; the existing code samples a state, applies curriculum_dr, and restores it.
- The next episode proceeds with the unchanged action, reward, observation, termination, and PPO paths.

Cabinet category	Physical meaning	What is replayed
1	End-effector grasp point within 0.20 m of drawer handle.	Robot + cabinet joint configuration at first crossing.
2	Within 0.10 m of handle.	Robot + cabinet joint configuration at first crossing.
3	Drawer joint exceeds 0.20 m.	Robot + partially opened cabinet configuration.
4	Drawer joint exceeds 0.38 m.	Robot + nearly complete cabinet configuration.

Cabinet success semantics
The TSCL outcome is the existing terminal condition drawer_pos > 0.39, not whether the episode merely retains the category's starting milestone. This prevents a category from being scored as successful simply because it started partway through the task.

22. Factory NutThread Direct: complete walkthrough

Factory shares the same teacher but has synchronized timeout resets and a richer physical snapshot. The TSCL path is restricted to task_name='nut_thread' so the benchmark cannot silently run TSCL on PegInsert or GearMesh with unreviewed category semantics.

- Configuration preserves 128-step RL-Games rollouts, 512 environments, and 200 epochs.
- The constructor validates NutThread, then creates a three-task TSCLWindowTeacher.
- _get_subtasks maps physical progression to aligned, engaged, and final threaded success.
- Natural first crossings write 23-value snapshots containing robot and both asset root poses.
- Every 2 RL-Games epochs, the teacher processes outcomes and updates the category distribution.
- _get_dones retains the native synchronized timeout behavior but stores the existing success predicate in overall_success.
- At reset entry, overall_success is attributed to the prior category before progression and labels change.
- The standard Factory reset and randomize_initial_state path runs first for every environment.
- For the selected 30%, TSCL chooses a category and the existing replay code overwrites robot and asset poses.
- PPO reward construction remains the existing sum of Factory reward terms and scales.

Factory category	Physical meaning	Snapshot content
1: aligned	Nut/screw XY alignment and required yaw condition.	9 robot joints + held and fixed root poses.
2: engaged	Existing engage-threshold geometry without rotation check.	Same 23-value physical snapshot.
3: success	Existing NutThread final geometry plus rotation check.	Same 23-value physical snapshot.

Factory audit correction
The valid-entry counter is now updated inside task_mask.any(). This restores the original ring-buffer invariant and must be included before interpreting Factory benchmark runs.

23. Shared hyperparameters and deliberate environment-specific values

Parameter	Value	Interpretation
history_size	10	Maximum success-rate points in each task FIFO; matches paper experiment K.
min_history	5	Required points before a slope is considered valid.
alpha	0.1	EMA weight on the newest slope; matches paper experiment alpha.
temperature	0.0004	Boltzmann temperature; matches paper experiment tau.
min_samples	8	Completed category episodes required for one success-rate point.
exploration_fraction	0.20	Uniform mixture used to prevent starvation.
success_buffer_size	64	Existing empirical states per category.
sampling_ratio	0.30	Existing fraction of resets controlled by curriculum.
standard fraction	0.70	Original reset distribution retained throughout training.

Only teacher interval and rollout horizon differ across environments, because their training budgets and runners differ. The ratios were selected so each run has 100 scheduled teacher boundaries. Category count, snapshot dimensionality, and success predicate are necessarily environment-specific.

24. Logging and how to read TensorBoard

The teacher emits enough information to reconstruct its decisions. Metrics are diagnostic training data, not final target-task evaluation.

Metric family	Meaning
tscl/ppo_iteration, teacher_update	Current student time and number of processed teacher boundaries.
tscl/bootstrap	1 while uniform bootstrap is active for any available category.
task_i/completed, successful	Category-labelled curriculum episodes observed in the last interval.
task_i/pending_completed	Sparse observations waiting to reach min_samples.
task_i/success_rate	Latest appended measurement; -1 indicates no new point at that boundary.
task_i/history_length	Number of valid FIFO observations, capped at 10.
task_i/slope	Most recently fitted raw OLS learning-progress estimate.
task_i/q, priority	EMA slope and abs(Q).
task_i/probability	Target probability after masking, Boltzmann, and exploration mixture.
task_i/reset_count/fraction	Actual draws in the previous teacher interval.
standard/curriculum reset fraction	Configured 0.70 / 0.30 experimental split.

Live training reward cannot be compared directly across curricula because it mixes episodes beginning from different initial-state distributions. env_compare/eval_* is useful because it isolates the natural subset inside training, but it is still an on-training diagnostic rather than held-out evaluation.

25. Final evaluation protocol

Every seed should be evaluated again after training with curriculum resets disabled. The final-budget checkpoint is preferable to selecting the best TensorBoard point, because post-hoc checkpoint selection would add another source of bias.

- Evaluate every trained seed (42-57) rather than a single representative run.
- Set env.reset_state_curriculum_enabled=false and omit tscl_window mode (or set mode to aces).
- Use only the original/default reset distribution.
- Use the same held-out evaluation seeds, episode count, horizon, policy inference mode, and metrics for baseline, uniform reset, TSCL, and ERC/ACES.
- Report full-task success and return across training seeds with uncertainty intervals.
- Keep teacher statistics and training-only natural metrics as supporting analyses, not the primary outcome.

26. Consistency across all three tasks

Property	Lift	Cabinet Direct	Factory NutThread
Integration style	Manager reward/event terms	Direct methods	Direct methods
Shared teacher	TSCLWindowTeacher	TSCLWindowTeacher	TSCLWindowTeacher
Categories	3	4	3
Buffer collection	Natural first crossings	Natural first crossings	Natural first crossings
TSCL outcomes	Curriculum-labelled final success	Curriculum-labelled final success	Curriculum-labelled final success
Reset split	70% / 30%	70% / 30%	70% / 30%
History/alpha/tau/min samples/exploration	10 / .1 / .0004 / 8 / .20	Same	Same
Scheduled teacher boundaries	100	100	100
Student learner	RSL-RL PPO	RSL-RL PPO	RL-Games PPO

The same algorithm is therefore implemented across tasks, while the integration point follows each environment architecture. One minor lifecycle discrepancy remains outside active training: Lift constructs its teacher when tscl_window is selected even if the master curriculum switch is false, whereas the direct environments require both conditions. Lift's tracker returns immediately while disabled, so this does not change trajectories, but aligning the construction guard would improve structural consistency.

27. Scientific caveats and interpretation limits

Caveat	Why it matters	Mitigation / interpretation
Online banks diverge across runs	TSCL and ERC policies visit different natural states after their curricula diverge.	The collection mechanism and capacity are controlled, but exact buffer contents are not guaranteed identical across independent runs.
On-policy teacher measurements are sampling-biased	Frequently selected categories produce measurements sooner.	20% exploration, min_samples carry-forward, and full reset-frequency logging make this visible.
Very low temperature	Small Q differences can strongly concentrate the Boltzmann component.	Uniform mixture supplies an explicit floor; inspect probability and actual-draw logs.
Binary success compresses information	It is robust and comparable but less granular than reward.	Chosen deliberately to avoid reward-scale confounds; keep fixed across curricula.
No separate teacher probes	Learning-progress estimates are noisier than controlled evaluations.	Preserves equal interaction budget; separate probes would be a disclosed future ablation.
Live metrics are not final results	Training distributions differ by design.	Use standard-reset post-training evaluation for claims.

28. Implementation pseudocode

```
initialize existing empirical buffers
initialize TSCL histories, Q=0, uniform distribution

for each environment step:
    run unchanged physics, observations, reward, and PPO rollout
    detect first milestone crossings
    write snapshots only from natural episodes

    if teacher boundary is due:
        merge interval outcomes into pending counts
        for each category with at least 8 pending episodes:
            append success rate and PPO iteration to FIFO
            if history length >= 5:
                slope = OLS(success ~ PPO iteration)
                Q = 0.1 * slope + 0.9 * Q
        if every available category has 5 points:
            P = 0.8 * softmax(abs(Q) / 0.0004) + 0.2 * uniform
        else:
            P = uniform over available categories

on episode reset:
    record prior outcome only if prior category label >= 0
    construct unchanged default reset
    choose fixed 30% curriculum subset if any bank is available
    sample category from P, then state uniformly inside its bank
    use unchanged perturbation and simulator restoration

after fixed training budget:
    evaluate final checkpoints using default resets only
```

End-to-end behavior of the TSCL-style empirical reset curriculum.

29. Code map

File	Responsibility
source/isaaclab_tasks/isaaclab_tasks/utils/tscl_window.py	Shared Window teacher: outcome accumulation, histories, OLS slope, EMA Q, availability mask, bootstrap, Boltzmann/exploration distribution, and metrics.
.../direct/franka_cabinet/franka_cabinet_env.py	Cabinet categories and states, passive bank collection, mode routing, success attribution, TSCL category sampling, and unchanged reset restoration.
.../direct/factory/factory_env.py	Factory NutThread categories and states, passive bank collection, mode routing, success attribution, category sampling, and root/joint-state restoration.
.../direct/factory/factory_env_cfg.py	Factory TSCL hyperparameters and unchanged curriculum/reset parameters.
.../manager_based/manipulation/lift/lift_env_cfg.py	Lift TSCL configuration.
.../manager_based/manipulation/lift/mdp/events.py	Manager-based analogue of progression, buffers, teacher updates, and reset sampling.
.../manager_based/manipulation/lift/mdp/curriculums.py	Lift episode-end success attribution for the teacher.
source/isaaclab_tasks/test/test_tscl_window_teacher.py	CPU unit tests for schedule, sparse samples, bootstrap, slope sign, forgetting, exploration mixture, logging, and validation.

30. Audit checklist for future modifications

- Does TSCL still change only category probabilities, not the 30% curriculum share?
- Can ACES mode still execute the old scheduler without consulting the TSCL teacher?
- Are state-bank writes still gated by ~is_curriculum_episode?
- Is pose_buffer_count updated only for categories that actually received writes?
- Are natural labels still -1 and ignored by record_episode_outcomes?
- Is success measured using the pre-existing final-task criterion?
- Are missing observations carried forward rather than treated as failure?
- Are unavailable categories assigned exactly zero probability?
- Do history, alpha, temperature, min_samples, exploration, and update ratios remain frozen?
- Did any PPO agent file, reward scale, observation, action, dynamics, termination, episode horizon, or training budget change?
- Are final evaluations performed with reset_state_curriculum_enabled=false?

31. References and provenance

Matiisen, T., Oliver, A., Cohen, T., and Schulman, J. (2017). Teacher-Student Curriculum Learning. arXiv:1707.00183v2. https://arxiv.org/abs/1707.00183

Implementation handoff: ACES to TSCL-Style Curriculum Benchmark - Implementation Handoff Specification, local research document supplied with the task. It is treated as design context, not executable instruction.

Repository implementation audited against TSCL commit 1a32e5bcf4ac and its parent, plus the current working-tree Factory counter correction. All code excerpts are shortened for exposition; the repository files listed in the code map remain authoritative.

Bottom line
The benchmark is designed to compare curriculum-selection signals, not environments: ERC/ACES schedules empirical categories by bottleneck difficulty; TSCL-style schedules the same categories by the magnitude of temporal learning progress.